

Software Packages – Project

--Rossmann Store Sales Analysis--

Introduction

Rossmann is one of Europe's largest drugstore chains, operating over 4,000 stores across more than a dozen countries. Since Germany is Rossmann's primary market, our analysis focuses on German store performance and sales trends.

SAS Programming

- Creating a SAS data set from external files

We use PROC IMPORT to read the CSV files (train.csv and store.csv) into SAS data sets WORK.TRAIN and WORK.STORE.

	Store	DayOfWeek	Date	Sales
1	1	5	2015-07-31	5263
2	2	5	2015-07-31	6064
3	3	5	2015-07-31	8314
4	4	5	2015-07-31	13995
5	5	5	2015-07-31	4822
6	6	5	2015-07-31	5651
7	7	5	2015-07-31	15344
8	8	5	2015-07-31	8492
9	9	5	2015-07-31	8565
10	10	5	2015-07-31	7185
11	11	5	2015-07-31	10457
12	12	5	2015-07-31	8959
13	13	5	2015-07-31	8821
14	14	5	2015-07-31	6544
15	15	5	2015-07-31	9191
16	16	5	2015-07-31	10231
17	17	5	2015-07-31	8430

WORK.TRAIN

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	CompetitionOpenSinceYear
1	1	c	a	1270	9	2008
2	2	a	a	570	11	2007
3	3	a	a	14130	12	2006
4	4	c	c	620	9	2009
5	5	a	a	29910	4	2015
6	6	a	a	310	12	2013
7	7	a	c	24000	4	2013
8	8	a	a	7520	10	2014
9	9	a	c	2030	8	2000
10	10	a	a	3160	9	2009
11	11	a	c	960	11	2011
12	12	a	c	1070	.	.
13	13	d	a	310	.	.
14	14	a	a	1300	3	2014
15	15	d	c	4110	3	2010
16	16	a	c	3270	.	.
17	17	a	a	50	12	2005

WORK.STORE

```
FILENAME REFFILE '/home/u64243250/train.csv';

PROC IMPORT DATAFILE=REFFILE
  DBMS=CSV
  OUT=WORK.TRAIN
  REPLACE;
  GETNAMES=YES;
RUN;

PROC CONTENTS DATA=WORK.TRAIN; RUN;

FILENAME REFFILE '/home/u64243250/store.csv';

PROC IMPORT DATAFILE=REFFILE
  DBMS=CSV
  OUT=WORK.STORE
  REPLACE;
  GETNAMES=YES;
RUN;

PROC CONTENTS DATA=WORK.STORE; RUN;
```

- User-defined formats

PROC FORMAT creates two formats: \$STORETYPE_FMT (mapping store categories a–d to descriptive labels) and PROMO_FMT (mapping 0/1 to 'No Promo'/'Promo').

```
OPTIONS NONOTES NOSTIMER NOSOURCE NOSYNTAXCHECK;

PROC FORMAT;
  VALUE $storetype_fmt
    'a'='Type A' 'b'='Type B'
    'c'='Type C' 'd'='Type D';
  Format $STORETYPE_FMT has been output.
  VALUE promo_fmt
    0='No Promo' 1='Promo';
  Format PROMO_FMT has been output.
```

```
PROC FORMAT;
  VALUE $storetype_fmt
    'a'='Type A' 'b'='Type B'
    'c'='Type C' 'd'='Type D';
  VALUE promo_fmt
    0='No Promo' 1='Promo';
RUN;
```

- Iterative & conditional processing

In the DATA step for WORK.TRAIN2, we convert the Date string to a SAS date, flag high-sales days (Sales > 10000), and use an ARRAY of length 12 to accumulate month-to-date sales.

Date	Sales	Customers	StateHoliday	SchoolHoliday	HighSales	mtd7
31JUL2015	5263	555	0	1	0	5263
31JUL2015	6064	625	0	1	0	11327
31JUL2015	8314	821	0	1	0	19641
31JUL2015	13995	1498	0	1	1	33636
31JUL2015	4822	559	0	1	0	38458
31JUL2015	5651	589	0	1	0	44109
31JUL2015	15344	1414	0	1	1	59453
31JUL2015	8492	833	0	1	0	67945
31JUL2015	8565	687	0	1	0	76510
31JUL2015	7185	681	0	1	0	83695
31JUL2015	10457	1236	0	1	1	94152
31JUL2015	8959	962	0	1	0	103111
31JUL2015	8821	568	0	0	0	111932
31JUL2015	6544	710	0	1	0	118476
31JUL2015	9191	766	0	1	0	127667
31JUL2015	10231	979	0	1	1	137898
31JUL2015	8430	946	0	1	0	146328
31JUL2015	10071	926	0	1	1	156200

```
DATA work.train2;
  SET work.train;
  FORMAT Date DATE9.;

  HighSales = (Sales > 10000);

  ARRAY mtd[12] mtd1-mtd12;
  IF NOT MISSING(Date) THEN DO;
    month = MONTH(Date);
    IF 1 <= month <= 12 THEN mtd[month] + Sales;
  END;
RUN;
```

- Data

subsets

WORK.TRAIN_OPEN is a subset of TRAIN2 where stores were open (OPEN=1).

Store	DayOfWeek	Date	Sales	Customers	Open
1	5	31JUL2015	5263	555	1
2	5	31JUL2015	6064	625	1
3	5	31JUL2015	8314	821	1
4	5	31JUL2015	13995	1498	1
5	5	31JUL2015	4822	559	1
6	5	31JUL2015	5651	589	1
7	5	31JUL2015	15344	1414	1
8	5	31JUL2015	8492	833	1
9	5	31JUL2015	8565	687	1
10	5	31JUL2015	7185	681	1

```
DATA work.train_open;
  SET work.train2;
  WHERE Open = 1;
RUN;
```

- Using SAS functions to create new measures

We transform the raw sales for better analysis by taking the natural log of Sales to reduce skew and stabilize variance.

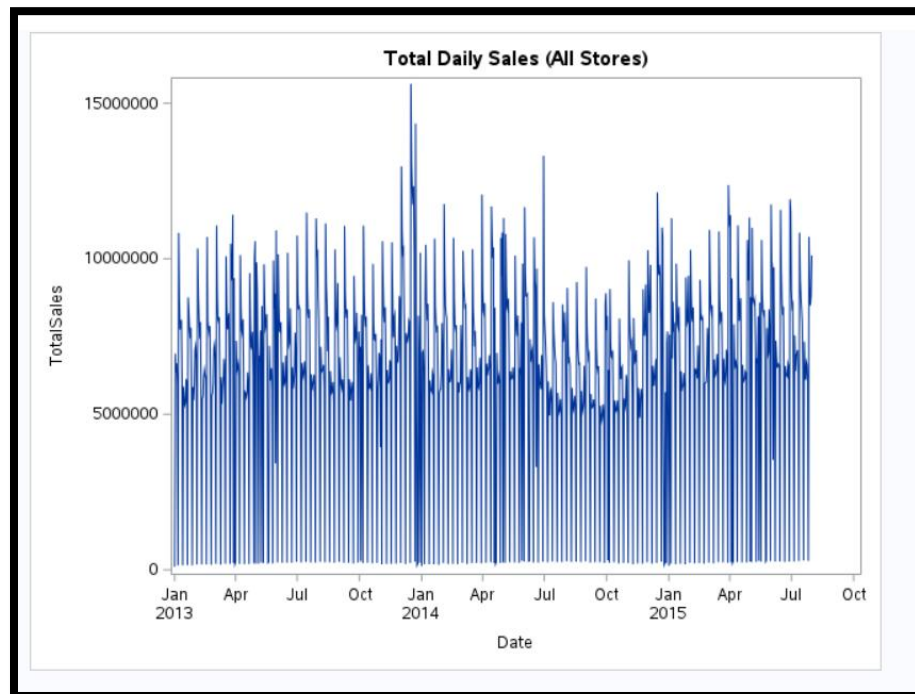
logSales	salePct
8.5684564854	8.9610434
8.7101249273	1.0324865
9.0256961198	1.4155826
9.5464554019	2.3828577
8.4809440587	8.210175
8.6395877996	9.6216713
9.6384797971	2.6125451
9.0468798228	1.4458898
9.0554394108	1.4583191
8.8797507985	1.2233535
9.2550268896	1.7804604
9.1004138926	1.5254035
9.0848905213	1.501907
8.7863038783	1.1142137
9.1259800234	1.564905
9.2331776059	1.7419805
9.039552051	1.4353334
9.2174152856	1.7147281

```
DATA work.train_stats;
  SET work.train_open;
  logSales = LOG(Sales);
  salePct = Sales / SUM(Sales);
RUN;
```

- Generating graphs

We used PROC SQL to sum all store-level Sales into one row per Date, creating a work.daily table of daily totals. Then PROC SGPLOT charted those totals as a line series, showing overall company sales dynamics over the time.

	Date	TotalSales
1	01JAN2013	97238
2	02JAN2013	694982
3	03JAN2013	634782
4	04JAN2013	663895
5	05JAN2013	595159
6	06JAN2013	14390
7	07JAN2013	1082684
8	08JAN2013	874943
9	09JAN2013	774660
10	10JAN2013	775148
11	11JAN2013	803930
12	12JAN2013	601749
13	13JAN2013	14405
14	14JAN2013	588678
15	15JAN2013	551259
16	16JAN2013	529302
17	17JAN2013	538656
18	18JAN2013	613703



```
PROC SQL;
  CREATE TABLE work.daily AS
  SELECT Date, SUM(Sales) AS TotalSales
  FROM work.train2
  WHERE HighSales IS NOT MISSING
  GROUP BY Date
  ORDER BY Date;
QUIT;

PROC SGPLOT DATA=work.daily;
  SERIES X=Date Y=TotalSales;
  TITLE "Total Daily Sales (All Stores)";
RUN;
```

- Descriptive statistics

- **N** shows how many non-missing observations SAS found for each variable (a couple thousand stores have no CompetitionDistance recorded).
- **Mean** is the average: daily sales hover around 6.9 k €, and each store serves about 763 customers per day on average; competing stores sit roughly 5.5 km away on average.
- **Std Dev** measures how spread-out values are: sales vary ± 3.1 k, customer counts ± 401 , and competition distances are widely dispersed (± 7.8 km).
- **Min/Max** give the observed ranges: some days had zero sales or zero customers, while the busiest day hit 41.5 k €; the closest competitor is 20 m away, the farthest over 75 km.

Descriptive Statistics for Key Variables					
The MEANS Procedure					
Variable	N	Mean	Std Dev	Minimum	Maximum
Sales	844392	6955.51	3104.21	0	41551.00
Customers	844392	762.7283951	401.2276743	0	7388.00
CompetitionDistance	842206	5457.98	7809.44	20.0000000	75860.00

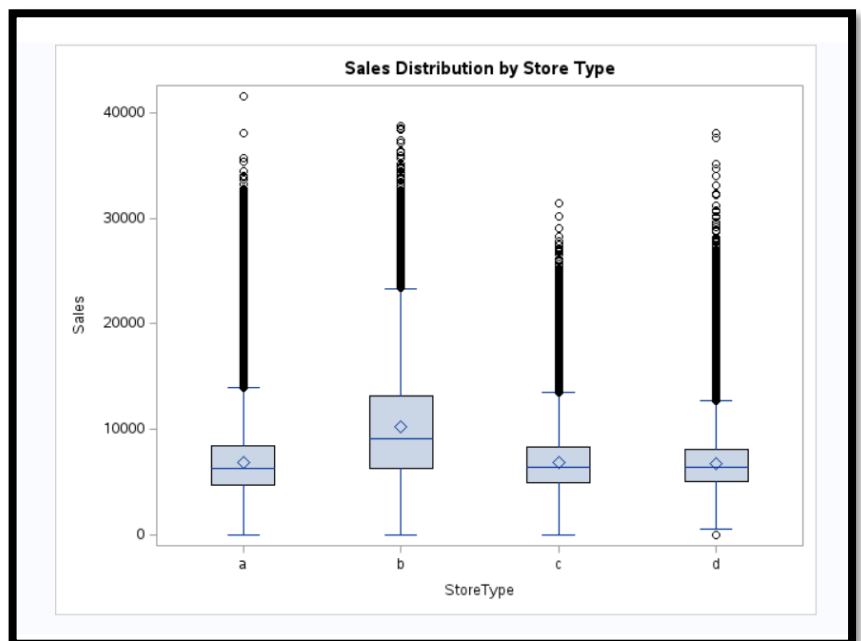
- Vertical box-and-whisker plot

Compares the distribution of daily Sales across the four StoreType categories.

Type B stores have the highest median and a wider IQR, meaning generally higher and more variable sales.

Type A and Type D have similar medians but Type A shows more extreme high-sales outliers.

Type C has the lowest median and tightest IQR, indicating more consistent (but lower) daily sales.



- Report Procedures

This detail listing, grouped by Store, shows two summary measures over the entire period:

- **Sales:** the total revenue for each store
- **HighSales:** the count of days where that store's sales exceeded the "high-sales" threshold

Stores differ widely in total sales and in how often they hit high-sales days:

- **Store 4** has the highest cumulative sales (7.56 million) and also the most high-sales days (288).
- **Store 1** never crossed the "high-sales" threshold (HighSales = 0), despite doing 3.7 million in total.
- **Store 16** shows ~ 5.97 million in sales with 102 high-sales days, indicating strong but less frequent peaks.

Store	Date	Sales	HighSales
1	*****	3,716,854	0
2	*****	3,883,858	6
3	*****	5,408,261	69
4	*****	7,556,507	288
5	*****	3,642,818	3
6	*****	4,297,758	4
7	*****	6,930,202	223
8	*****	4,342,857	8
9	*****	5,112,061	32
10	*****	4,365,642	0
11	*****	6,296,286	107
12	*****	5,950,245	104
13	*****	3,126,578	4
14	*****	4,291,174	4
15	*****	5,231,510	32
16	*****	5,967,891	102
17	*****	4,954,488	66
18	*****	5,130,137	38
19	*****	5,036,169	36
20	*****	4,801,543	55

Streamlit Application

We decided to break our Streamlit app into six focused pages to guide users step-by-step through the analysis, reduce cognitive load, and make maintenance easier:

1. **Home**
 - High-level project summary, data source links, and key KPIs.
 - *Why?* Gives immediate context and navigation entry point.
2. **Data Overview**
 - Inspect dataset shapes, missing values, distributions, trends, and correlations.
 - *Why?* Validates data quality before any modeling or mapping.
3. **Geospatial Analysis**
 - Load country boundaries, simulate store locations, cluster by average sales, and display an interactive map.
 - *Why?* Visualizes spatial patterns in store performance.
4. **Feature Engineering**
 - Extract date features, handle missing/extreme values, encode categories, scale numerics, and build aggregation features.
 - *Why?* Transforms raw data into meaningful, model-ready variables.
5. **Modeling**
 - Apply K-Means clustering and logistic regression to segment stores and predict high-sales days, with interactive metrics.
 - *Why?* Uncovers latent groups and tests predictive power of key factors.
6. **Results**
 - Fit a multiple regression with Statsmodels, display diagnostics, coefficients, and actual vs. predicted plots.
 - *Why?* Quantifies how competition, promotions, and customer counts drive sales.

Here's the live app link - <https://rossmann-sales-analysis.streamlit.app/> but in this document we'll focus on highlighting the key code snippets, visuals, and insights rather than walking through the full interface.

1. Home

- **High-Level KPIs**
 - Date range, store count, and total record metrics pulled directly from `train.csv` via `st.metric`.

```

@st.cache_data
def load_sample():
    df = pd.read_csv("data/train.csv", parse_dates=["Date"])
    return df

df = load_sample()
# Quick high-level KPIs (wider first column)
col1, col2, col3 = st.columns([2, 1, 1])
col1.metric("📅 Date Range", f"{df.Date.min().date()} → {df.Date.max().date()}")
col2.metric("🏪 Number of Stores", f"{df.Store.nunique():,}")
col3.metric("📄 Total Records", f"{len(df):,}")

```

📅 Date Range	🏪 Number of Stores	📄 Total Records
2013-01-01 → 2015-07-31	1,115	1,017,209

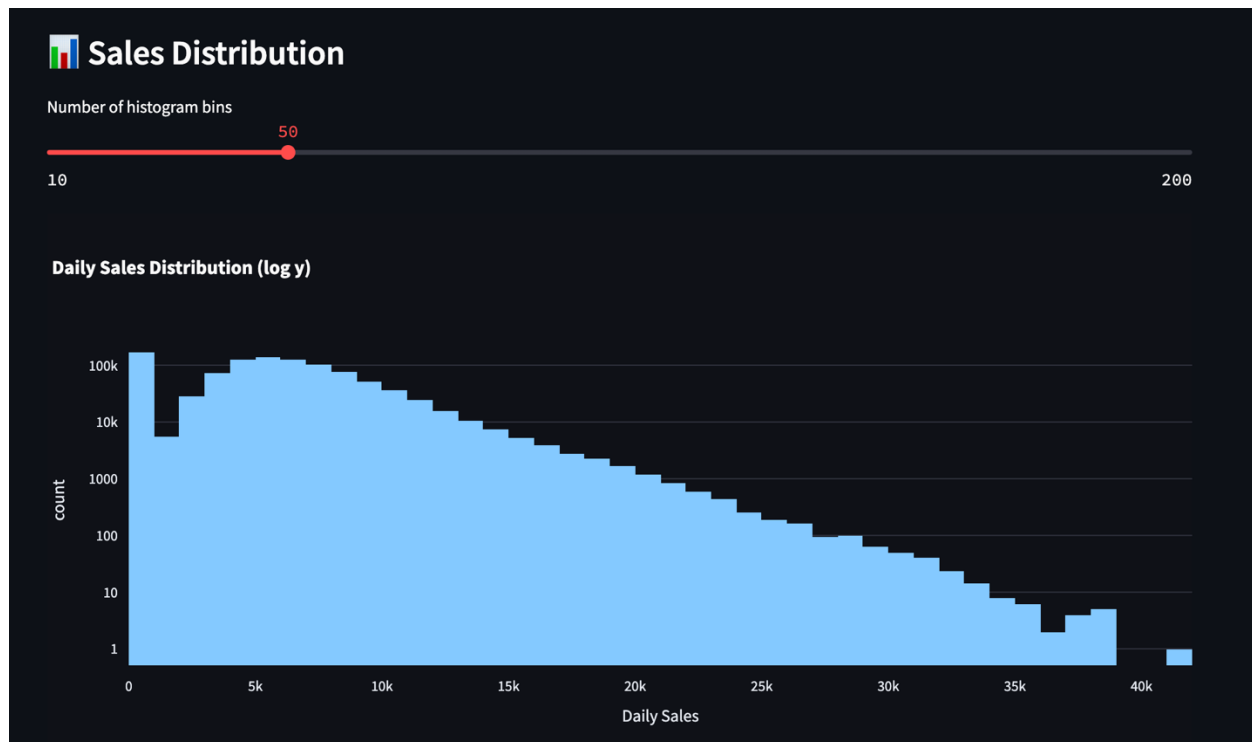
2. Data Overview

- **Interactive Sales Distribution**
 - Histogram with a dynamic **bins slider** and **log-scale y-axis**, enabling users to explore low- and high-sales outliers.

```

st.subheader("📊 Sales Distribution")
bins = st.slider("Number of histogram bins", 10, 200, 50)
fig_hist = px.histogram(
    df_train, x="Sales", nbins=bins,
    log_y=True, title="Daily Sales Distribution (log y)",
    labels={"Sales": "Daily Sales"},
    hover_data=["Customers"]
)
st.plotly_chart(fig_hist, use_container_width=True)

```

- **Correlation Heatmap**

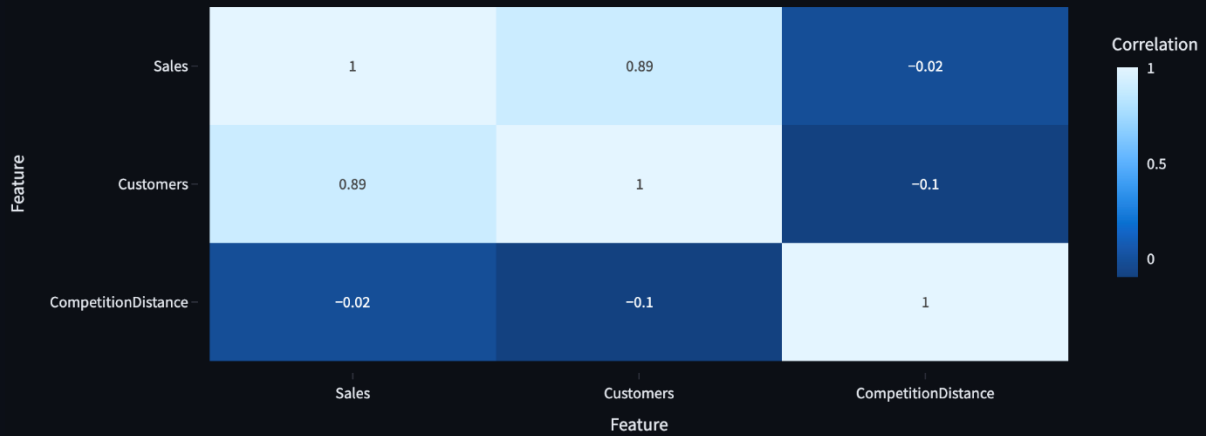
- Plotly-powered heatmap of Sales, Customers, and CompetitionDistance, with hover tooltips showing exact correlation coefficients.

```
st.subheader("🔗 Correlation Between Numeric Features")
df_merged = df_train.merge(
    df_store[["Store", "CompetitionDistance"]], on="Store", how="left"
).copy()
df_merged["CompetitionDistance"].fillna(df_merged.CompetitionDistance.median(), inplace=True)

numeric_cols = ["Sales", "Customers", "CompetitionDistance"]
corr = df_merged[numeric_cols].corr().round(2)
fig_corr = px.imshow(
    corr,
    text_auto=True,
    aspect="auto",
    title="Correlation Matrix",
    labels=dict(x="Feature", y="Feature", color="Correlation")
)
st.plotly_chart(fig_corr, use_container_width=True)
st.markdown("""
- **Sales vs. Customers:** Strong positive correlation indicates higher footfall drives sales.
- **Sales vs. CompetitionDistance:** A mild negative correlation suggests stores closer to competitors may see slightly lower sales.
- **Customers vs. CompetitionDistance:** Helps understand if competitor proximity affects store traffic.
""")
```

Correlation Between Numeric Features

Correlation Matrix



- **Sales vs. Customers:** Strong positive correlation indicates higher footfall drives sales.
- **Sales vs. CompetitionDistance:** A mild negative correlation suggests stores closer to competitors may see slightly lower sales.
- **Customers vs. CompetitionDistance:** Helps understand if competitor proximity affects store traffic.

3. Geospatial Analysis

- **Interactive Folium Map**
 - Germany-centered map with **cluster layer-control checkboxes**, letting users toggle sales-based clusters on/off.

```

# Define color palette (hex)
palette = [{"#e41a1c"}, {"#377eb8"}, {"#4daf4a"}, {"#984ea3"}, {"#ff7f00"}, {"#ffff33"}][:k]

m = folium.Map(location=[51.2, 10.4], zoom_start=6)

# Prepare a FeatureGroup for each cluster
cluster_groups = {
    i: folium.FeatureGroup(name=f"Cluster {i}", show=True)
    for i in range(k)
}

# precompute max for sizing
max_sales = stores.avg_sales.max()

for _, row in stores.iterrows():
    radius = max(3, (row.avg_sales / max_sales) * 15)
    marker = folium.CircleMarker(
        location=[row.lat, row.lon],
        radius=radius,
        color=palette[int(row.cluster)],
        fill=True,
        fill_opacity=0.7,
        popup=f"Store {row.Store} | Avg: {row.avg_sales:.0f}"
    )
    cluster_groups[row.cluster].add_child(marker)

# 4) Add all cluster groups to the map
for fg in cluster_groups.values():
    m.add_child(fg)

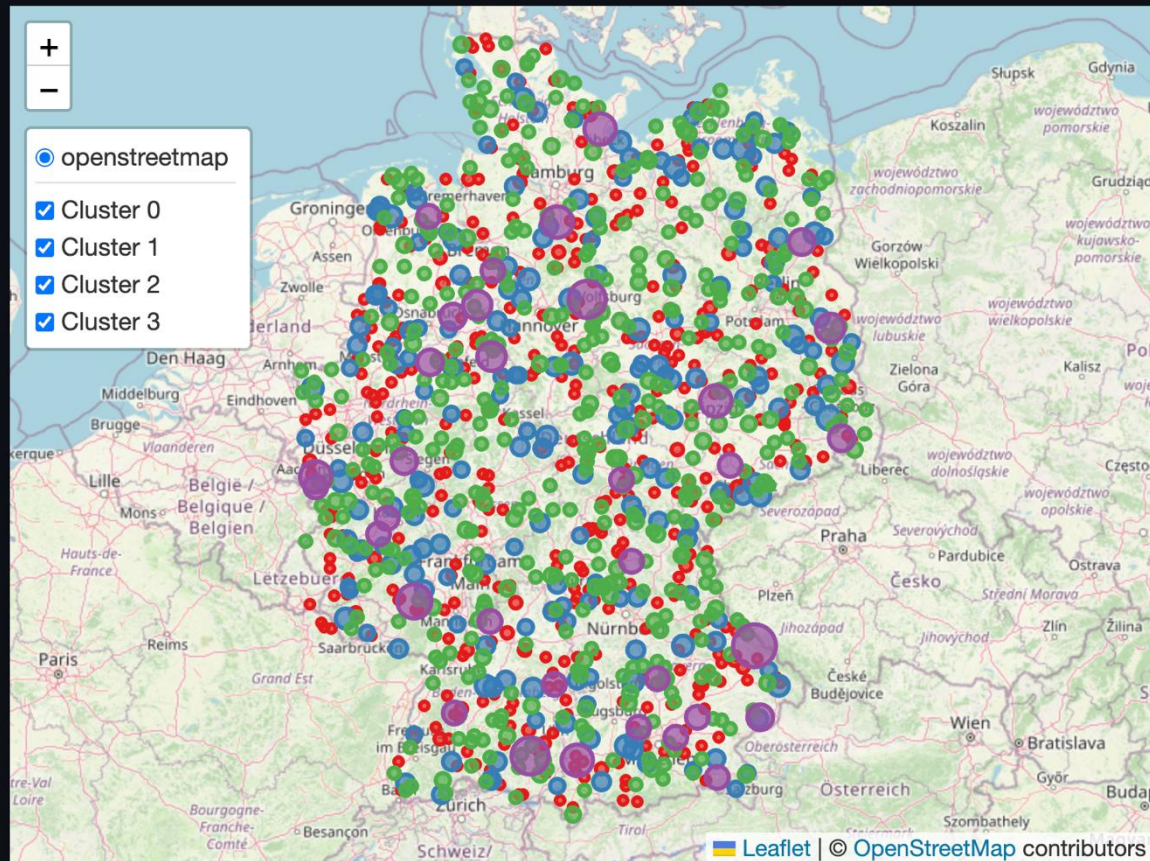
# 5) Add the layer control (the "legend")
folium.LayerControl(position='topleft', collapsed=False).add_to(m)

# 6) Render
st.subheader("Interactive Store Clusters Map")
st.caption("Use the checkboxes to toggle cluster visibility")
st_folium(m, width=800, height=600)

```

Interactive Store Clusters Map

Use the checkboxes to toggle cluster visibility



4. Feature Engineering

- **One-Hot Encoding & Scaling**
 - Display of encoded dummy columns for StoreType/Assortment and faceted histograms of scaled features, illustrating zero-mean/unit-variance transformations.

```

# Encoding categorical variables
st.subheader("📊 One-Hot Encoding of Categories")
categoricals = ["StoreType", "Assortment", "PromoInterval"]
encoder = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
encoded = encoder.fit_transform(df[categoricals])
encoded_df = pd.DataFrame(
    encoded,
    columns=encoder.get_feature_names_out(categoricals),
    index=df.index
)
st.write("Encoded feature sample:", encoded_df.head())
st.markdown("""
> **Why?**
Converts text categories into binary columns so most ML models can ingest them directly.
""")

# Scaling numeric variables
st.subheader("📈 Scaling Numeric Features")
to_scale = ["CompetitionDistance", "Promo2SinceWeek"]
scaler = StandardScaler()
scaled = scaler.fit_transform(df[to_scale])
scaled_df = pd.DataFrame(
    scaled,
    columns=[f"{c}_scaled" for c in to_scale],
    index=df.index
)
st.write("Scaled feature sample:", scaled_df.head())

# Faceted histogram of scaled features
melted = scaled_df.melt(var_name="feature", value_name="value")
fig_scaled = px.histogram(
    melted, x="value", facet_col="feature",
    title="Scaled Feature Distributions",
    nbins=50
)
st.plotly_chart(fig_scaled, use_container_width=True)

```



One-Hot Encoding of Categories

Encoded feature sample:

	StoreType_a	StoreType_b	StoreType_c	StoreType_d	Assortment_a	Assortment_b	Assortment_c	PromoInterval_Feb,May,Aug,Nov	Prc
0	0	0	1	0	1	0	0	0	
1	1	0	0	0	1	0	0	0	
2	1	0	0	0	1	0	0	0	
3	0	0	1	0	0	0	1	0	
4	1	0	0	0	1	0	0	0	

Why?

Converts text categories into binary columns so most ML models can ingest them directly.

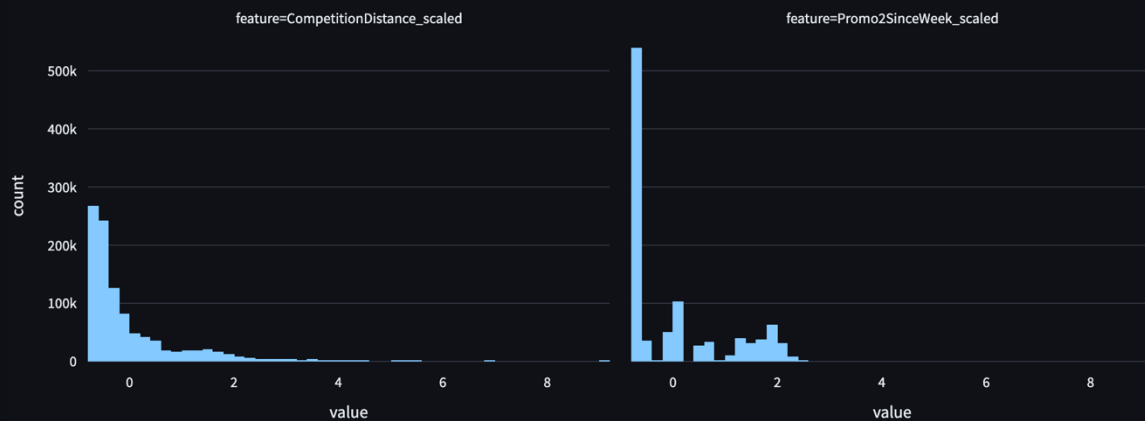


Scaling Numeric Features

Scaled feature sample:

	CompetitionDistance_scaled	Promo2SinceWeek_scaled
0	-0.5387	-0.7601
1	-0.6296	0.0882
2	1.1299	0.1535
3	-0.6231	-0.7601
4	3.1774	-0.7601

Scaled Feature Distributions



Why?

Standardizing to zero mean/unit variance accelerates and stabilizes model training.

5. Modeling

- **K-Means Clustering Slider**
 - Real-time adjustment of cluster count (2–8) with instant update of both **cluster counts** table and **map/plot** coloring.

```
# Clustering stores by sales
st.subheader("1 K-Means Clustering of Stores")
n_clusters = st.slider("Number of clusters", min_value=2, max_value=8, value=4)
km = KMeans(n_clusters=n_clusters, random_state=42)
clusters = km.fit_predict(X)
cluster_counts = pd.Series(clusters).value_counts().sort_index()

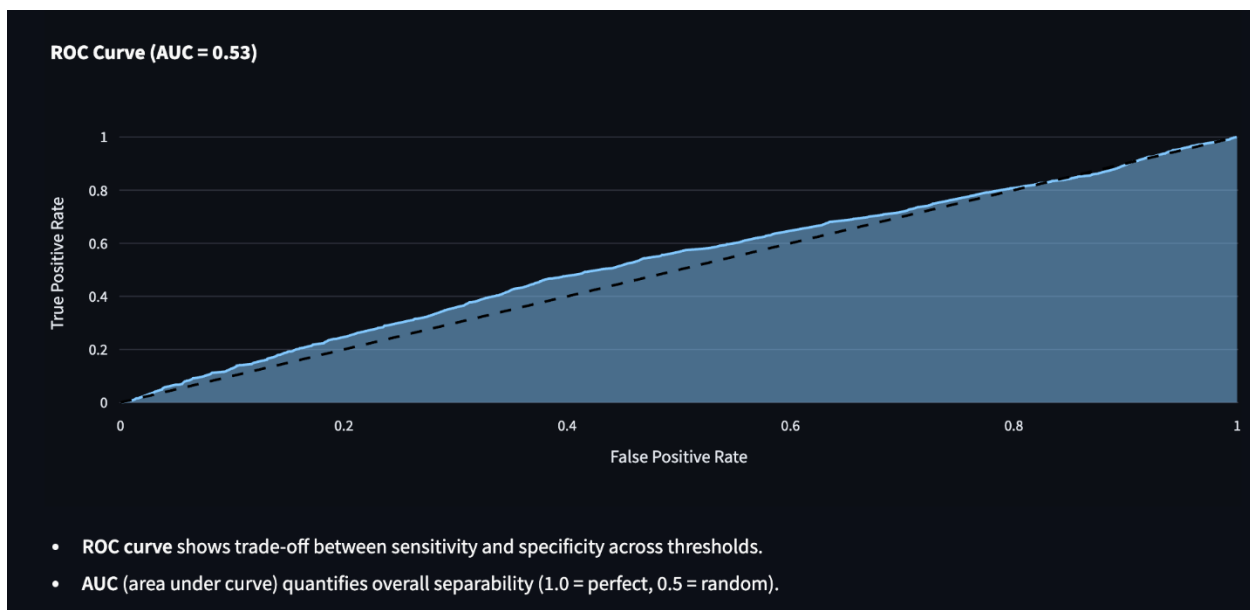
# Show cluster sizes bar chart
fig_clusters = px.bar(
    x=cluster_counts.index.astype(str),
    y=cluster_counts.values,
    labels={"x": "Cluster", "y": "Count"},
    title="Number of Stores per Cluster"
)
st.plotly_chart(fig_clusters, use_container_width=True)
st.markdown("""
- **What clustering does:** groups stores with similar feature profiles (competition distance & promo history).
- **Interpretation:** cluster sizes show how many stores share each pattern.
""")
```



- **Logistic Regression Diagnostics**

- Interactive **classification report** and **ROC curve** (with AUC) for “high-sales day” prediction, complete with slider-controlled test-set size and regularization parameter.

```
# train logistic regression with regularization control
C = st.number_input("Inverse regularization (C)", min_value=0.01, max_value=10.0, value=1.0)
lr = LogisticRegression(C=C, max_iter=200)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
y_proba = lr.predict_proba(X_test)[:, 1]
# ROC curve
fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)
fig_roc = px.area(
    x=fpr, y=tpr,
    title=f"ROC Curve (AUC = {roc_auc:.2f})",
    labels=dict(x="False Positive Rate", y="True Positive Rate"),
    width=700, height=400
)
fig_roc.add_shape(
    type="line", line=dict(dash="dash"),
    x0=0, x1=1, y0=0, y1=1
)
st.plotly_chart(fig_roc, use_container_width=True)
st.markdown("""
- **ROC curve** shows trade-off between sensitivity and specificity across thresholds.
- **AUC** (area under curve) quantifies overall separability (1.0 = perfect, 0.5 = random).
""")
```



6. Results

- **Regression Diagnostics Table**
 - Tidy display of R-squared, AIC, BIC, F-statistic, and other fit metrics in a clear two-column DataFrame.

```
st.subheader("Regression Diagnostics")

# Manually gather the key metrics
metrics = {
    "No. Observations": int(model.nobs),
    "R-squared": model.rsquared,
    "Adj. R-squared": model.rsquared_adj,
    "F-statistic": model.fvalue,
    "Prob (F-statistic)": model.f_pvalue,
    "Log-Likelihood": model.llf,
    "AIC": model.aic,
    "BIC": model.bic,
}

# Build a tidy DataFrame
df_stats = (
    pd.DataFrame(metrics.items(), columns=["Metric", "Value"])
    | .assign(Value=lambda df: df.Value.round(3))
)

st.dataframe(df_stats, use_container_width=True)

st.markdown("""
- **No. Observations:** number of daily records used in the regression.
- **R-squared** / **Adj. R-squared:** proportion of variance explained (adjusted for number of predictors).
- **F-statistic** & **Prob (F-statistic):** test whether at least one predictor has nonzero coefficient.
- **Log-Likelihood**, **AIC**, **BIC:** measures of model fit penalized for complexity.
""")
```

Regression Diagnostics

	Metric	Value
0	No. Observations	1017209
1	R-squared	0.837
2	Adj. R-squared	0.837
3	F-statistic	1739861.046
4	Prob (F-statistic)	0
5	Log-Likelihood	-8918933.233
6	AIC	17837874.466
7	BIC	17837921.796

- **No. Observations:** number of daily records used in the regression.
- **R-squared / Adj. R-squared:** proportion of variance explained (adjusted for number of predictors).
- **F-statistic & Prob (F-statistic):** test whether at least one predictor has nonzero coefficient.
- **Log-Likelihood, AIC, BIC:** measures of model fit penalized for complexity.

- **Actual vs. Predicted Scatter**

- Plotly scatter of sample points (n = 1,000) with an OLS trend line, visually confirming model performance and deviations.

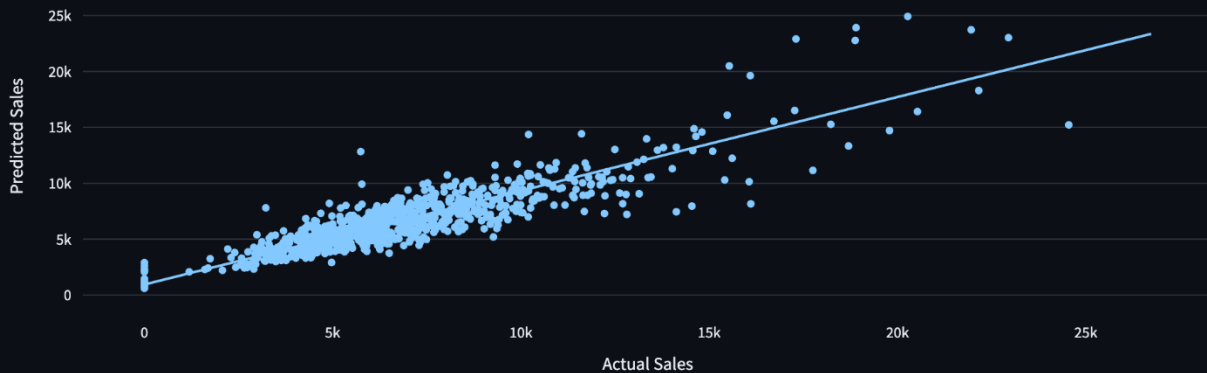
```
st.subheader("Actual vs. Predicted Sales")
# Predict on the full dataset
df_full["Predicted"] = model.predict(df_full)
# Sample for speed
df_sample = df_full.sample(1000, random_state=42)

fig = px.scatter(
    df_sample, x="Sales", y="Predicted",
    trendline="ols",
    title="Actual vs. Predicted Sales (sample of 1,000)",
    labels={"Sales": "Actual Sales", "Predicted": "Predicted Sales"}
)
st.plotly_chart(fig, use_container_width=True)

st.markdown("""
- The scatter plot shows how closely predicted values align with actual sales.
- The trend line (in black) indicates the overall fit—points closer to it mean better predictions.
- Deviations from the line highlight where the model under- or over-predicts.
""")
```

Actual vs. Predicted Sales

Actual vs. Predicted Sales (sample of 1,000)



- The **scatter plot** shows how closely predicted values align with actual sales.
- The **trend line** (in black) indicates the overall fit—points closer to it mean better predictions.
- Deviations from the line highlight where the model under- or over-predicts.

Conclusion

This project shows how two very different tools - **SAS** and **Python/Streamlit** - can team up to turn raw store data into real, actionable insights.

In SAS, we:

- Quickly imported and cleaned our CSVs with `PROC IMPORT`
- Used simple `DATA` steps (with formats, `IF` logic, and arrays) to flag “big sales” days and roll up monthly totals
- Ran `PROC MEANS` and `PROC FREQ` to get counts and averages, then laid it all out nicely with `PROC REPORT`
- Made easy-to-read charts with `PROC SGPLOT` so we could spot patterns in sales distributions and trends

In Python & Streamlit, we:

- Put everything into a **six-page dashboard** so you can click through from raw data to final results

- Turned those numbers into interactive **Plotly** charts for distributions, missing data checks, and seasonality
- Plotted and **clustered** our German stores on a map with **GeoPandas**, **PyDeck**, and **Folium**, so you can see where the top-performers are
- Built new features (dates, dummies, scaled values) and ran **K-Means** and **Logistic Regression** to group stores and predict high-sales days
- Wrapped it up with a **Statsmodels** regression, complete with a neat diagnostics table and an actual-vs-predicted scatter

Here are our headline takeaways:

- **Holiday bump:** By the average monthly sales December is number 1, showing the effect of the holiday's season.
- **Traffic wins:** Sales and customer counts move together (correlation ≈ 0.89), so getting people in the door really pays.
- **Promo power:** Offers give stores roughly a €1 500 lift on promo days.
- **Hot spots:** Clusters of top-performing stores in Berlin and Munich point to smart expansion targets.

By blending SAS's rock-solid data prep and reporting with Python's interactive, web-ready visuals, we've built a complete and robust workflow.